

# Security Review

Cantina Managed

**ALTO MONEY: V1**

Cantina Managed review by:

**Phaze**, Lead Security Researcher



September 26, 2026



# Contents

|          |                                                                                                     |          |
|----------|-----------------------------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                                                                 | <b>2</b> |
| 1.1      | About Cantina . . . . .                                                                             | 2        |
| 1.2      | Disclaimer . . . . .                                                                                | 2        |
| 1.3      | Risk assessment . . . . .                                                                           | 2        |
| 1.3.1    | Severity Classification . . . . .                                                                   | 2        |
| <b>2</b> | <b>Security Review Summary</b>                                                                      | <b>3</b> |
| 2.1      | Scope . . . . .                                                                                     | 3        |
| <b>3</b> | <b>Findings</b>                                                                                     | <b>4</b> |
| 3.1      | Low Risk . . . . .                                                                                  | 4        |
| 3.1.1    | Terminal repayment can leave borrow shares that block credit-account rotation . . . . .             | 4        |
| 3.1.2    | Minting all pending fees before repayment can block debt reduction below fore-closure LTV . . . . . | 6        |
| 3.2      | Informational . . . . .                                                                             | 7        |
| 3.2.1    | Permissionless collateral donations can front-run credit-account rotation . . . . .                 | 7        |
| 3.2.2    | Zero-interest accrual emits events unlike the Mint market . . . . .                                 | 8        |
| 3.2.3    | Borrow-token exclusion prevents forwarding DUSD rewards . . . . .                                   | 8        |



# 1 Introduction

## 1.1 About Cantina

Cantina is a security platform that pairs a world-class network of security researchers with agentic AI to help teams building onchain find and fix real risk. Through managed reviews, competitions, and bounties, Cantina secures protocols before vulnerabilities can be exploited.

Learn more at [cantina.security](https://cantina.security)

## 1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

## 1.3 Risk assessment

| Severity level     | Impact: High | Impact: Medium | Impact: Low |
|--------------------|--------------|----------------|-------------|
| Likelihood: high   | Critical     | High           | Medium      |
| Likelihood: medium | High         | Medium         | Low         |
| Likelihood: low    | Medium       | Low            | Low         |

### 1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).



## 2 Security Review Summary

Alto is a decentralized credit protocol built around DUSD, focused keeping risk contained by isolating markets, using a single stablecoin and partial liquidations.

From Sep 14th to Sep 16th the Cantina team conducted a review of [v1](#) on commit hash [438ad257](#). The team identified a total of **5** issues:

Issues Found

| Severity          | Count    | Fixed    | Acknowledged |
|-------------------|----------|----------|--------------|
| Critical Risk     | 0        | 0        | 0            |
| High Risk         | 0        | 0        | 0            |
| Medium Risk       | 0        | 0        | 0            |
| Low Risk          | 2        | 1        | 1            |
| Gas Optimizations | 0        | 0        | 0            |
| Informational     | 3        | 3        | 0            |
| <b>Total</b>      | <b>5</b> | <b>4</b> | <b>1</b>     |

### 2.1 Scope

The security review had the following components in scope for [v1](#) on commit hash [438ad257](#):

```
contracts
├── lending
│   ├── credit-market
│   │   ├── AltoConvexCreditLineLib.sol
│   │   └── AltoConvexCreditLineMarket.sol
│   └── libraries
│       ├── AssetShareConversionMath.sol
│       └── Uint128Converter.sol
└── utils
    ├── AltoOwnable2StepUpgradeable.sol
    ├── AuthUpgradeable.sol
    ├── Constants.sol
    ├── FixedPointMath.sol
    └── Pausable.sol
```



## 3 Findings

### 3.1 Low Risk

#### 3.1.1 Terminal repayment can leave borrow shares that block credit-account rotation

**Severity:** Low Risk.

**Context:** `AltoConvexCreditLineMarket.sol`#L165-L205.

**Description:** An attacker can use permissionless `repay()` to burn the credit account's entire outstanding DUSD debt while leaving a residual borrow share. The remaining `position[creditAccount].borrowShares` blocks credit-account rotation and cannot be cleared through further repayment or collateral additions alone.

1. Alice, the legitimate credit account, deposits 1,000 LP tokens and borrows 100 DUSD.
2. Bob, an unrelated attacker, calls `repay(0, position[creditAccount].borrowShares - 1, creditAccount)`. He pays the entire outstanding debt because upward rounding charges the full asset amount despite leaving one share unpaid.
3. `_repay()` subtracts the computed `assets` and requested `shares` independently, leaving `totalBorrowed.assets == 0` and `totalBorrowed.shares == position[creditAccount].borrowShares - 1`.

This occurs when `convertToAssetsUp(S - 1, A, S) == A`, where `A = totalBorrowed.assets` and `S = totalBorrowed.shares` at repayment. Once debt assets reach zero, any positive share repayment still rounds up to at least one DUSD base unit. Both share- and asset-denominated repayments therefore underflow at `totalBorrowed.assets -= assets.toUint128()`. Zero debt assets accrue no interest, so waiting does not clear the residual share.

At a 1:1 oracle, `maxLtv == 0.6 ether`, and `foreclosureLtv == 0.8 ether`, the residual share represents one rounded debt base unit. Alice can withdraw all but two LP base units, but `setCreditAccount()` requires both `borrowShares` and `collateralAssets` to be zero. The owner cannot use `governanceLiquidate()` to clean up these residual shares because the position's rounded debt does not exceed the foreclosure threshold. Mint markets share the repayment arithmetic but allow this administrative cleanup without a foreclosure threshold and have no credit-account rotation requirement. Recovery in the credit line remains possible through suitable risk-parameter changes followed by `governanceLiquidate()`, or an implementation upgrade.

**Impact explanation:** The attack blocks credit-account rotation even after all outstanding debt assets are repaid, requiring a recovery workaround to clear the position. The demonstrated collateral lock affects only base-unit dust, with no material fund loss established.

**Likelihood explanation:** Bob must fund the outstanding DUSD debt and target a position whose asset/share ratio satisfies the terminal-rounding condition. The funding requirement limits practicality for large debts but becomes inexpensive when the remaining debt is small.

**Recommendation:** When the nonzero computed repayment exactly consumes all remaining `totalBorrowed.assets`, consider consuming all remaining account and global borrow shares while burning the same full asset amount.

**Proof of concept:** The following proof of concept demonstrates a one-base-unit debt instance of the terminal-share state and blocked ordinary recovery operations.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import {stdError} from "forge-std/StdError.sol";
```



```

import {AltoConvexCreditLineMarket} from
↳ "@alto/lending/credit-market/AltoConvexCreditLineMarket.sol";
import {
    ConvexCreditLineInitParams,
    IAltoConvexCreditLineMarketErrors
} from "@alto/lending/interfaces/IAltoConvexCreditLineMarket.sol";
import {IMarketErrors} from "@alto/lending/interfaces/IMarket.sol";

import {ConvexCreditLineCommon} from "../ConvexCreditLineCommon.t.sol";

contract PartialShareRepaymentPermissionlesslyStrandsTerminalBorrowSharesTest is
↳ ConvexCreditLineCommon {
    function test_PermissionlessPartialShareRepaymentStrandsTerminalBorrowShares()
    ↳ external {
        // Match the audited deployment's zero opening fee and 60%/80% LTV
        ↳ parameters.
        ConvexCreditLineInitParams memory params = _validInitParams();
        params.creditMarketInitParams.baseMarketInitParams.maxLtv = 0.6 ether;
        params.creditMarketInitParams.foreclosureLtv = 0.8 ether;
        params.creditMarketInitParams.initialBorrowOpeningFee = 0;
        market = _initialize(params);

        vm.startPrank(OWNER);
        borrowToken.setMinterStatus(address(market), true);
        borrowToken.setMinterCeiling(address(market), type(uint256).max);
        borrowToken.setBurnerStatus(address(market), true);
        vm.stopPrank();

        // Two LP base units provide exactly one DUSD base unit of borrowing
        ↳ capacity.
        _openPosition(2, 1);
        (uint128 debtAssetsBefore, uint128 debtSharesBefore) =
        ↳ market.totalBorrowed();
        assertEq(debtAssetsBefore, 1);
        assertEq(debtSharesBefore, 1_000_000);

        // Model normal token circulation: the attacker is merely an unrelated DUSD
        ↳ holder.
        vm.prank(CREDIT_ACCOUNT);
        borrowToken.transfer(USER, 1);
        assertEq(borrowToken.balanceOf(USER), 1);

        // No caller authorization is required. Repaying one share rounds its asset
        ↳ cost up to one,
        // independently subtracting all assets but only one of the account's one
        ↳ million shares.
        vm.prank(USER);
        (uint256 burnedAssets, uint256 repaidShares) = market.repay(0, 1,
        ↳ CREDIT_ACCOUNT);
        assertEq(burnedAssets, 1);
        assertEq(repaidShares, 1);
        assertEq(borrowToken.balanceOf(USER), 0);
        assertEq(borrowToken.totalSupply(), 0);

        (uint128 debtAssetsAfter, uint128 debtSharesAfter) = market.totalBorrowed();
        (, uint128 accountSharesAfter, uint128 collateralAfter) =
        ↳ market.position(CREDIT_ACCOUNT);
        assertEq(debtAssetsAfter, 0);
        assertEq(debtSharesAfter, debtSharesBefore - 1);
        assertEq(accountSharesAfter, debtSharesAfter);
        assertEq(collateralAfter, 2);
        assertGt(accountSharesAfter, 0);
        (uint256 marketMinted,) = borrowToken.minterConfig(address(market));
        assertEq(marketMinted, 0);
    }
}

```



```

// Positive time cannot repair the terminal ratio because interest on zero
→ assets is zero.
vm.warp(block.timestamp + 365 days);
market.accrueInterest();
(uint128 debtAssetsAfterWarp, uint128 debtSharesAfterWarp) =
→ market.totalBorrowed();
assertEq(debtAssetsAfterWarp, 0);
assertEq(debtSharesAfterWarp, debtSharesAfter);

// Neither repayment denomination can clear the shares: both try to subtract
→ at least one
// asset from the zero global asset balance before any further token burn
→ can occur.
vm.prank(USER);
vm.expectRevert(stdError.arithmeticError);
market.repay(0, accountSharesAfter, CREDIT_ACCOUNT);

vm.prank(USER);
vm.expectRevert(stdError.arithmeticError);
market.repay(1, 0, CREDIT_ACCOUNT);

// The virtual-share conversion still values the stranded shares as one debt
→ unit. Removing
// either collateral unit would leave zero max-LTV capacity, so the account
→ cannot empty.
vm.prank(CREDIT_ACCOUNT);
vm.expectRevert(IMarketErrors.AltoBaseMarketInsufficientUserCollateral.selector);
market.removeCollateral(1, CREDIT_ACCOUNT, CREDIT_ACCOUNT, "");

// Governance cannot rotate the account while debt/collateral remains, and
→ the default 80%
// foreclosure threshold is exactly one unit, while foreclosure requires
→ debt to exceed it.
vm.prank(OWNER);
vm.expectRevert(IAItoConvexCreditLineMarketErrors.AltoCreditMarketPositionN
→ otEmpty.selector);
market.setCreditAccount(makeAddr("ReplacementCreditAccount"));

vm.prank(OWNER);
vm.expectRevert(IAItoConvexCreditLineMarketErrors.AltoCreditMarketPositionN
→ otForeclosable.selector);
market.governanceLiquidate(CREDIT_ACCOUNT, false);

(uint128 finalAssets, uint128 finalShares) = market.totalBorrowed();
(uint128 supplyShares, uint128 finalAccountShares, uint128 finalCollateral)
→ = market.position(CREDIT_ACCOUNT);
assertEq(finalAssets, 0);
assertEq(finalShares, debtSharesAfter);
assertEq(supplyShares, 0);
assertEq(finalAccountShares, accountSharesAfter);
assertEq(finalCollateral, 2);
}
}

```

**Alto:** Fixed in [PR 658](#).

**Cantina Managed:** [PR 658](#) mitigates this by restricting repayment to the credit account, preventing third-party repayment griefing. The remaining full-repayment rounding limitation is acknowledged and accepted.

### 3.1.2 Minting all pending fees before repayment can block debt reduction below foreclosure LTV

**Severity:** Low Risk.



**Context:** `AltoConvexCreditLineMarket.sol#L400-L412`.

**Description:** `AltoConvexCreditLineMarket._repay()` mints all pending interest and fees before burning the payer's DUSD. If that mint exceeds the market's DUSD minter ceiling, every repayment reverts, even when the borrower has sufficient funds. Debt can reach this ceiling while remaining below the collateral-based foreclosure threshold.

The deployment uses a 1,000,000 DUSD credit limit, a 1,312,500 DUSD minter ceiling, 60% max LTV, 80% foreclosure LTV, and a fixed 1:1 LP oracle. Consider this scenario:

1. Alice deposits 1,666,667 LP and borrows 1,000,000 DUSD. Interest can increase debt above the credit limit, which restricts new borrowing.
2. Debt grows to 1,320,000 DUSD without repayments or fee claims. Alice holds sufficient DUSD and attempts to repay 100,000 DUSD.
3. `_repay()` first mints the 320,000 DUSD of pending interest. That would increase the market's outstanding minted amount from 1,000,000 to 1,320,000 DUSD, exceeding the ceiling by 7,500 DUSD. `DUSD.mint()` reverts before the repayment burn.
4. Debt is approximately 79.2% of collateral value, below the 80% foreclosure threshold of 1,333,333.6 DUSD. `governanceLiquidate()` therefore rejects foreclosure, while Alice cannot repay to reduce debt to 1,220,000 DUSD.

The ceiling formula, `creditLimit * 1.05 ether / foreclosureLtv`, leaves mint capacity below the foreclosure threshold even at minimum collateral. Additional collateral raises that threshold further. Fee claims do not solve the problem because they exchange pending fees for outstanding minted DUSD without reducing debt.

At 4% annual interest with frequent accrual, a full draw reaches the ceiling after approximately 6.8 years without repayment. Regular debt servicing prevents the scenario, and governance can restore repayment by raising the ceiling. The demonstrated consequence is temporary repayment unavailability without direct asset loss.

**Recommendation:** Consider allowing repayment without first minting every pending fee. Claim fees within available headroom and use repayment burns to free capacity, retaining unminted fees for later claims. Respect `DUSD.burn()`'s restriction that a market cannot burn more than its outstanding minted amount; larger payments require staged minting and burns or repayment in tranches. Simply reversing mint and burn does not handle every case.

As an immediate mitigation, increase the ceiling and monitor headroom alongside credit-limit, rate, and LTV changes. A 1,400,000 DUSD ceiling covers the minimum-collateral foreclosure threshold with 5% slack, but does not guarantee repayment for every collateral amount or accrual period.

**Alto:** Acknowledged.

**Cantina Managed:** Acknowledged without a code change. The mint-before-burn ordering is retained because reversing the order can exceed DUSD's per-market burn limit; governance can raise the minter ceiling when needed.

## 3.2 Informational

### 3.2.1 Permissionless collateral donations can front-run credit-account rotation

**Severity:** Informational.

**Context:** `AltoConvexCreditLineMarket.sol#L92-L95`.

**Description:** An outsider can repeatedly delay credit-account rotation in `AltoConvexCreditLineMarket` by donating one LP base unit before governance calls `setCreditAccount()`. Each donation makes the outgoing position nonempty, so rotation reverts with `AltoCreditMarketPositionNot Empty`.

1. After the outgoing credit account repays all debt and withdraws all collateral, Alice, the market owner, submits `setCreditAccount()` to replace it.



2. Bob front-runs Alice with `addCollateral(1, oldAccount)`, funding the donation himself.
3. Alice's transaction reverts. After the outgoing account withdraws the donated unit, Bob can repeat the attack against another rotation attempt.

`addCollateral()` permits unauthorized callers to deposit for the current credit account, while `setCreditAccount()` requires both its borrow shares and collateral assets to be zero. Withdrawing a donation restores that condition but does not prevent another donation before rotation.

The attack costs Bob one LP base unit plus gas per successful attempt and causes failed transactions and additional coordination. Bob gains no withdrawal authority and cannot steal collateral. This is recoverable operational griefing, not permanent DoS: freezing blocks deposits while allowing repayment, collateral withdrawal, and rotation.

**Recommendation:** Consider enforcing a freeze-before-cleanup rotation procedure: freeze, repay all borrow shares, withdraw the actual remaining collateral balance, rotate while frozen, then unfreeze. Full pause is not a substitute during cleanup because it blocks ordinary repayment and withdrawal.

**Alto:** Fixed in [PR 655](#).

**Cantina Managed:** [PR 655](#) restricts collateral deposits to the credit account, preventing third-party donations from blocking account rotation.

### 3.2.2 Zero-interest accrual emits events unlike the Mint market

**Severity:** Informational.

**Context:** [AltoConvexCreditLineMarket.sol#L439](#).

**Description:** `AltoConvexCreditLineMarket._accrueInterest()` emits `AccrueInterest` even when the interest-rate model returns zero interest. `AltoMintMarket._accrueInterest()` instead returns without emitting the event in that case.

For example, consecutive accrual calls within the same block can produce zero-interest events in the credit-line market. Indexers or monitoring systems that assume each event represents an increase in debt must handle this difference. The additional logs also consume gas, but they do not change debt or fee accounting.

Existing tests explicitly require the zero-interest event, so this is an event-semantics consistency observation rather than an accounting vulnerability.

**Recommendation:** Consider matching the Mint market by emitting only when interest is nonzero, or document the difference and ensure event consumers handle zero-interest checkpoints.

**Alto:** Fixed in [PR 657](#).

**Cantina Managed:** [PR 657](#) emits `AccrueInterest` only when interest is nonzero, matching the Mint market behavior.

### 3.2.3 Borrow-token exclusion prevents forwarding DUSD rewards

**Severity:** Informational.

**Context:** [AltoConvexCreditLineLib.sol#L166](#).

**Description:** `AltoConvexCreditLineLib.validateRewardTokenAddition()` rejects the market's borrow token, DUSD. Governance therefore cannot configure DUSD reward forwarding even if the upstream Convex pool introduces DUSD incentives.

If such an incentive program exists and pays DUSD to the market, `claimRewards()` requests upstream extra rewards but forwards only configured reward-token balances. DUSD cannot enter that list and remains in the market instead of reaching `rewardRecipient`. Ordinary repayment and bad-debt settlement burn tokens from the payer or owner, not the market's held balance, and the market exposes no DUSD sweep.

Existing tests explicitly enforce the borrow-token exclusion.



**Recommendation:** Consider confirming whether DUSD incentives must be supported. If so, provide a controlled path that transfers actual DUSD reward balances without minting tokens or changing debt accounting. Otherwise, document the unsupported reward asset and avoid DUSD incentive programs for this integration.

**Alto:** Fixed in [PR 659](#).

**Cantina Managed:** [PR 659](#) documents that DUSD rewards are intentionally unsupported; this integration will not use DUSD reward incentives.